

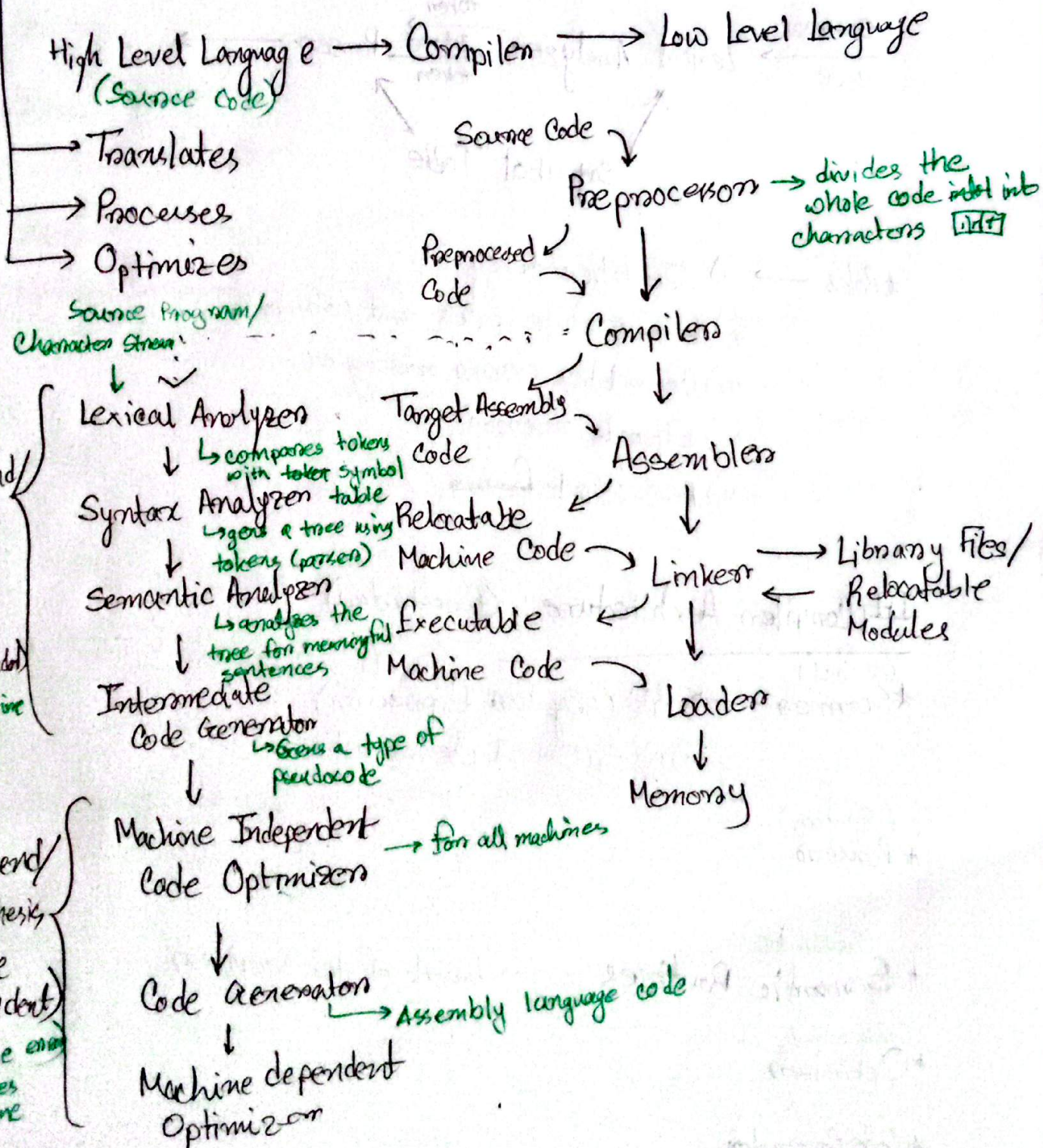
C-1, 2, (w-1) (a-1)

4c2jejaw

28/06/26

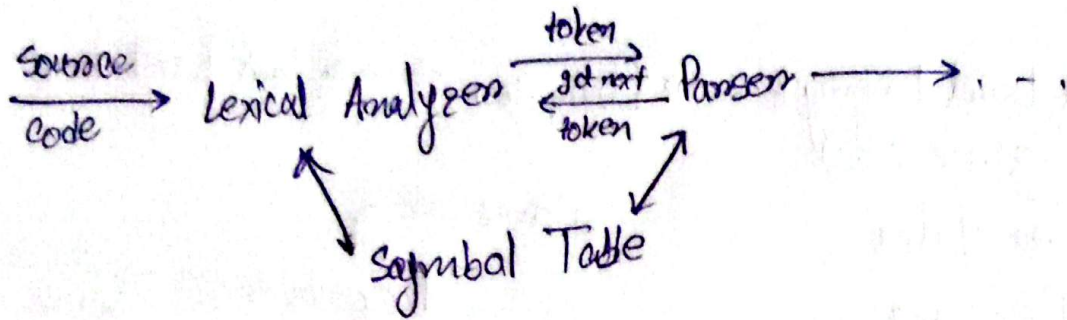


Compilers:



Lexical Analyzer:

* Lexeme: sequence of characters according to the matching pattern of a token



- * Roles →
- i) Identifies token
 - ii) Removes white spaces and comments
 - iii) Correlates error messages
 - iv) Expands macros
 - v) Read input ~~from~~

Compiler Architecture: (Foreshadaw)

(Lexical)
* Scanner → i) RE (Regular Expression)
ii) NFA (Non-Deterministic)

(Syntax)
* Parser

(Semantic)
* Semantic Routines → heart of the compiler

(Code optimizer)
* Optimizer

* Code Generator

Formal Language:

* Natural language is language without grammar
Formal " " " with grammar

* Basic Elements:

(i) Symbol $\rightarrow$ letters

(ii) Alphabet $\rightarrow$ set of symbols (Σ) $\{\}$

(iii) String/Word $\rightarrow$ sequence of symbols $\rightarrow \epsilon$ (empty string) $\emptyset$
 $\rightarrow | \cdot |$ (length)

$\rightarrow \emptyset$ if prefix/suffix = string,
it is called proper pre/suffix

$\rightarrow$ powers of alphabet = no. of
symbols per combination

$\rightarrow \Sigma^+ =$ all possible
sets

$\Sigma^+ =$ non empty sets

(iv) Languages $\rightarrow$ set of strings from $\Sigma^+ \rightarrow L \subseteq \Sigma^+$

25/06/20

+ Regular Expression Operations:

ii) CONCATENATION $\rightarrow \{ \cdot \} \rightarrow \{ \cdot \cdot \cdot \cdot \cdot \}$

$$* \vee \cdot \vee \cup$$

↓
No words

Length of word is 0

$$R = L\{0^* + 1\} \rightarrow 0^+ = \{e, 0, 00, \dots\}$$
$$\therefore R = \{e, 0, \infty, \dots\}$$

011, 110, ... }
L No. 1111

1. $R = L(0^* \cdot 11) \rightarrow$ will never have ϵ in result
 $= \{011, 0011, 00011, \dots\}$
 $\hookrightarrow$ No $\cdot 110$

$$L(0+1)^* = L(0^*+1^*)$$
$$\rightarrow L(0.1)^* = L(0^* \cdot 1^*) \quad \times$$

C-5 & 4 (w-2) (q-1)

30/06/26

(i)* $L(0+10^*)$

(ii)* $L(0^*10^*)$

(iii)* $L((0(0+1))^*)$

(iv)* Even length of string where $\Sigma = \{a, b\}$

(i) $R = L(0+10^*)$

Here, $10^* = \{\epsilon, 10, 1010, 101010, \dots\}$

$0 = 0$

$\therefore R = \{\epsilon, 0, 10, 01010, 0101010, \dots\}$

X 10^*
Not (10^*)

(ii) $L(0^*10^*)$

Here, $0^* = \{\epsilon, 0, 00, 000, \dots\}$

$10^* = \{\epsilon, 10, 1010, 101010, \dots\}$

$\therefore R = \{\epsilon, 0, 00, 000, \dots, 010, 01010, 0101010, \dots, 0010, 001010, \dots, 10, 1010, 101010, \dots\}$

X

(iii) $L(0(0+1)^*) = L((00+01)^*) = L(00^*+01^*)$

Here, $00^* = \{\epsilon, 00, 0000, 000000, \dots\}$

$01^* = \{\epsilon, 01, 0101, 010101, \dots\}$

$\therefore R = \{\epsilon, 0\}$

$$(i) R = L(0+10^*) \rightarrow L(0) \cup L(010^*)$$

$$\text{Here, } 0^* = \{\epsilon, 0, 00, 000, \dots\}$$

$$\therefore 10^* = \{1, 10, 100, 1000, \dots\}$$

$$\therefore R = L(0+10^*) = \{\overset{0}{0}, \overset{1}{01}, 010, 0100, 01000, \dots, 10, 100, 1000, 10000, \dots\}$$

$$(ii) R = L(0^*10^*) \rightarrow L(0^*) \cdot L(1) \cdot L(0^*)$$

$$\text{Here, } 0^* = \{\epsilon, 0, 00, \dots\}$$

$$\therefore 10^* = \{1, 10, 100, 1000, \dots\}$$

$$\therefore R = L(0^*10^*) = \{\overset{1}{01}, \overset{10}{010}, \overset{100}{0100}, 01000, \dots, 001, 0010, 00010, \dots\}$$

$$(iii) R = L((0(0+1))^*) \rightarrow L(0(L(0) \cup L(1)))^*$$

$$\text{Here, } (0+1) = \{01, 10\} \quad \left| \quad R = \{\epsilon, 00, 01, 000, 001, \dots, 010101, 010101, \dots, 0100, 01000, \dots\}$$

$$\therefore (0(0+1))^* = \{001, 010\}$$

$$\therefore (0(0+1))^* = \{\epsilon, 001010, 001010, 001001010, \dots, 001010010, 00101001010, \dots\}$$

$$\therefore R = L((0(0+1))^*) = \{\epsilon, 001, 001010, 001001010, \dots, 001010010, 00101001010, \dots\}$$

(iv) Given length of string where $\Sigma = \{a, b\}$

$$R = ((a+b)(a+b))^n$$

$$R = \{ \epsilon, aa, ab, ba, bb, aaaa, aaab, aabb, \dots \}$$

Reg from string: [Practice on ex 93]

DFA: Deterministic Finite Automata

state transitions are defined Discrete input $\rightarrow$ Discrete output

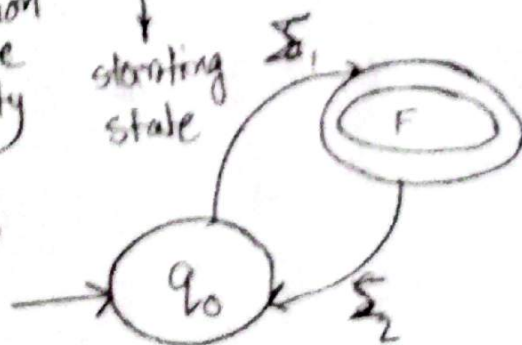
In exam write this

$$A = (Q, \Sigma, \delta, q_0, F)$$

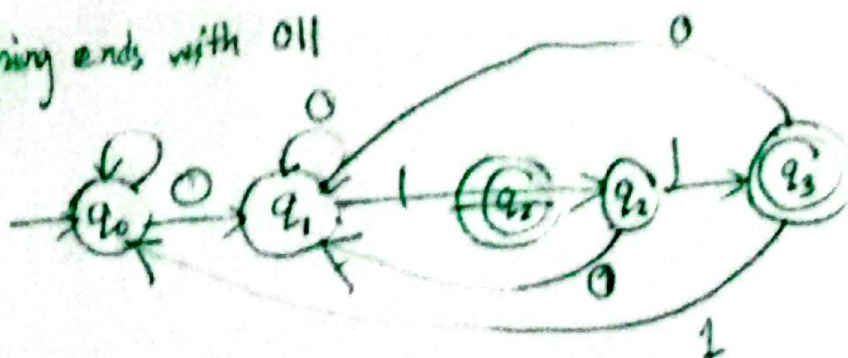
finite non empty set of states
transition function table
finite non empty input symbols and alphabets

starting state
Final state

+ DM PTSD
yes



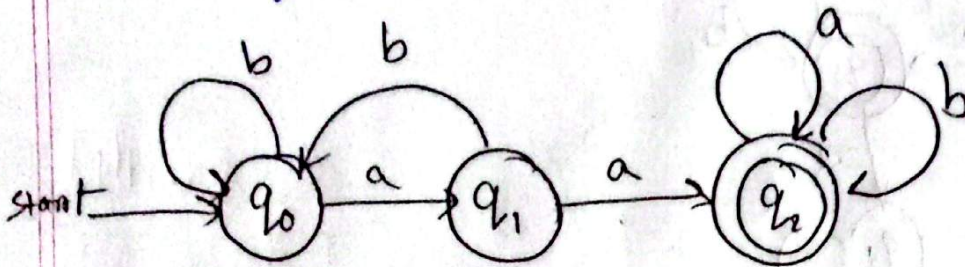
String ends with 011



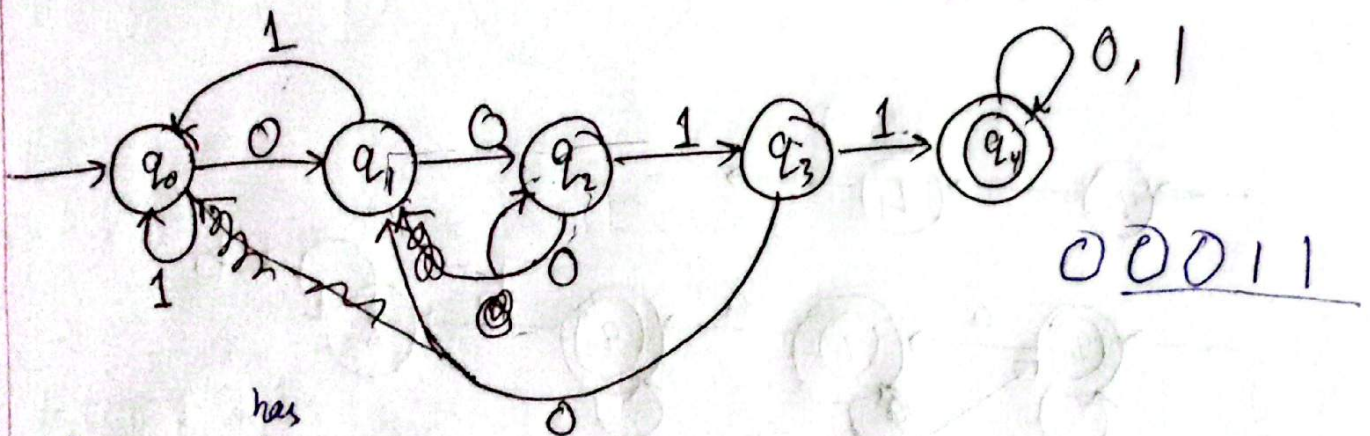
C-6, W-2 (Q-1)

02/07/26

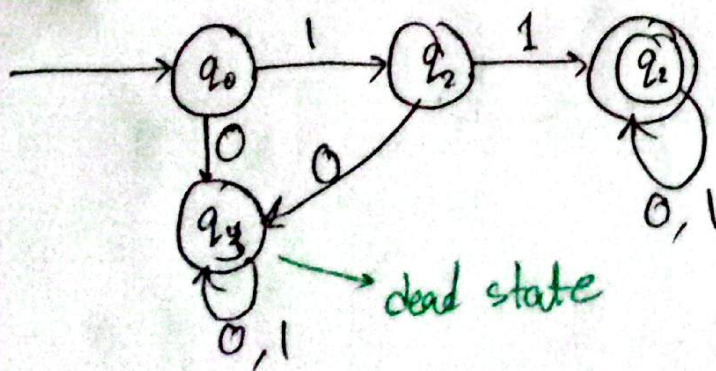
*Design a DFA to accept string a's and b's having substring aa



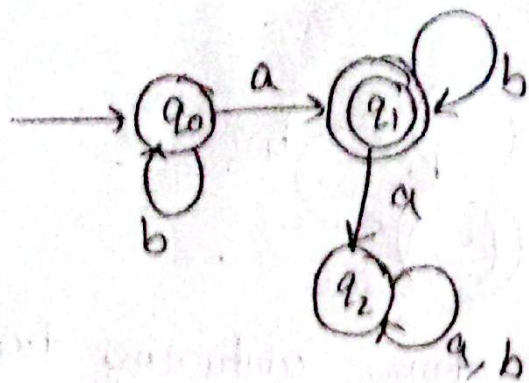
*Design a DFA to accept strings containing two consecutive 0's followed by two consecutive 1's



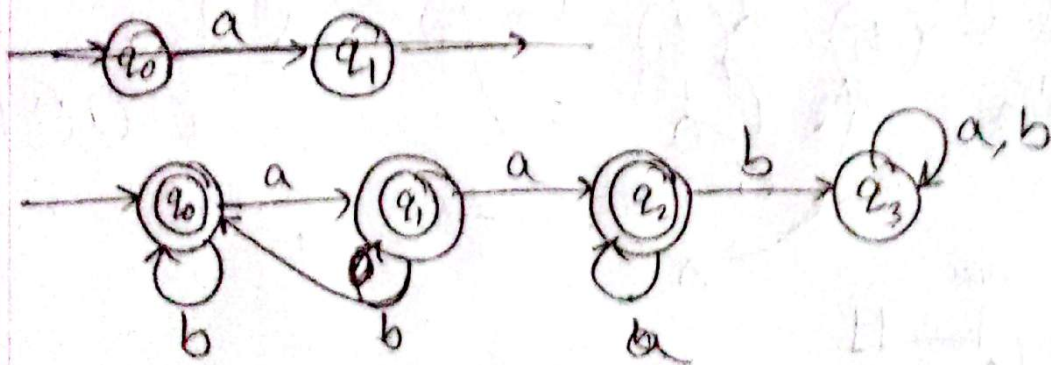
*Beginning has 11



* 'a' and 'b' with exactly one a



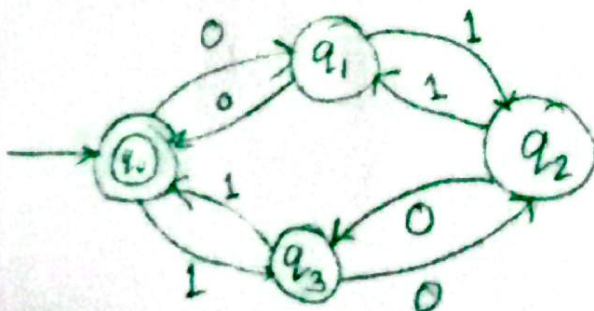
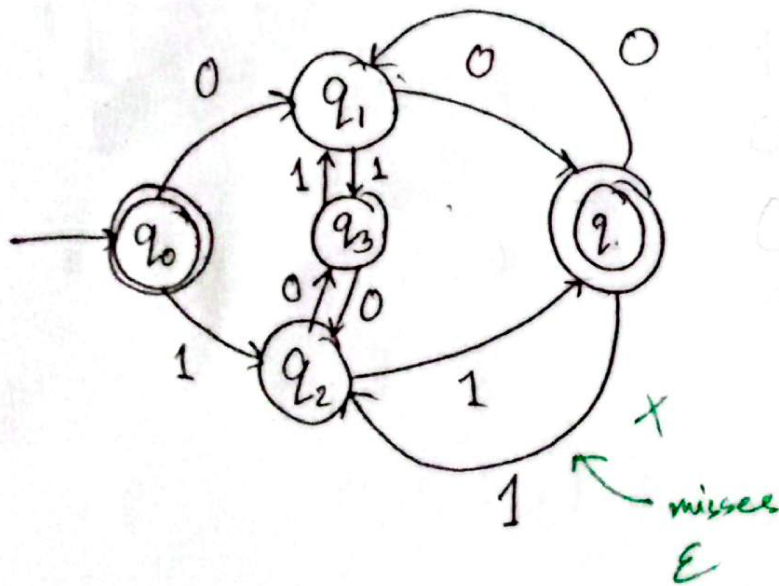
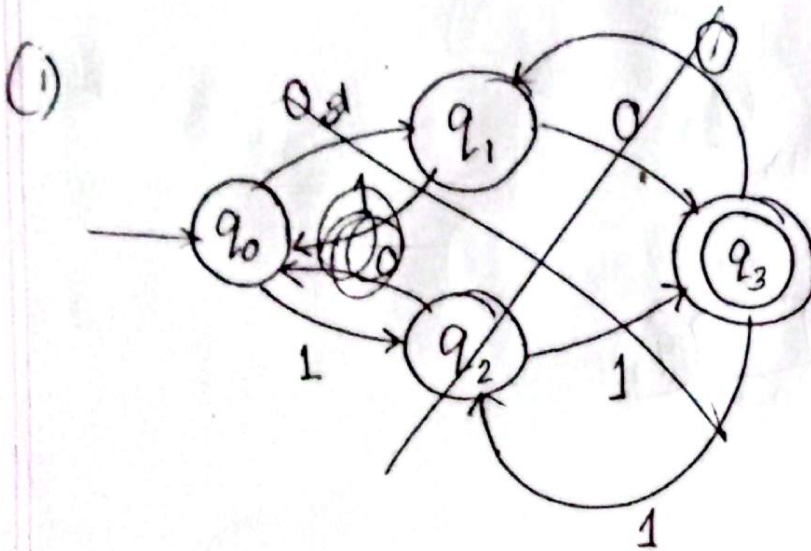
* 'a', 'b' without a a b



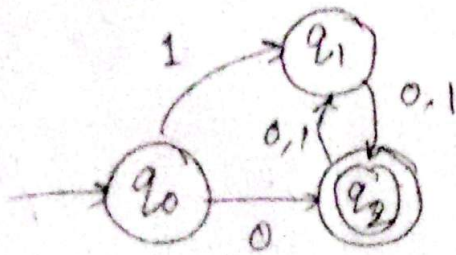
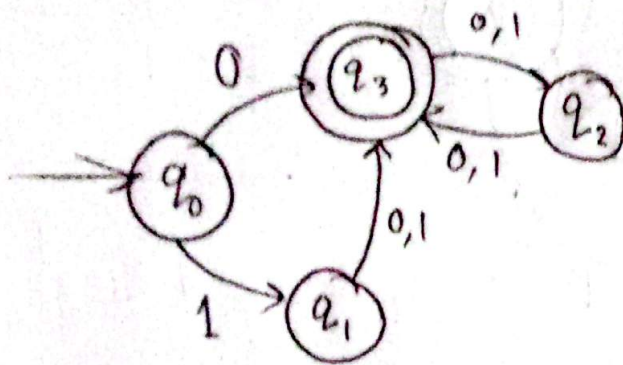
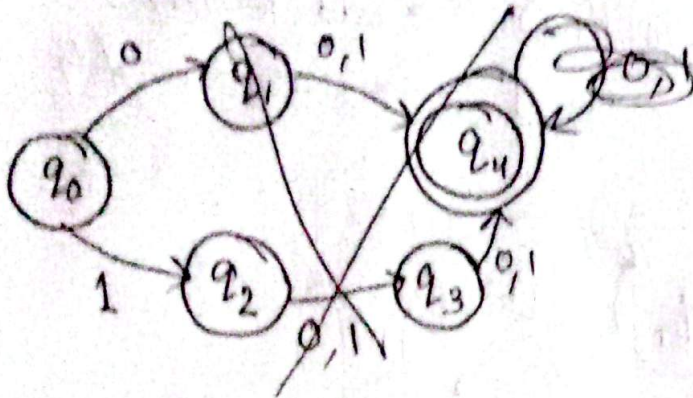
abbaab

abab

- HW: i) Even num of 0s and 1s
 ii) If starts with 0 $\rightarrow$ odd length
 " " " 1 $\rightarrow$ even length



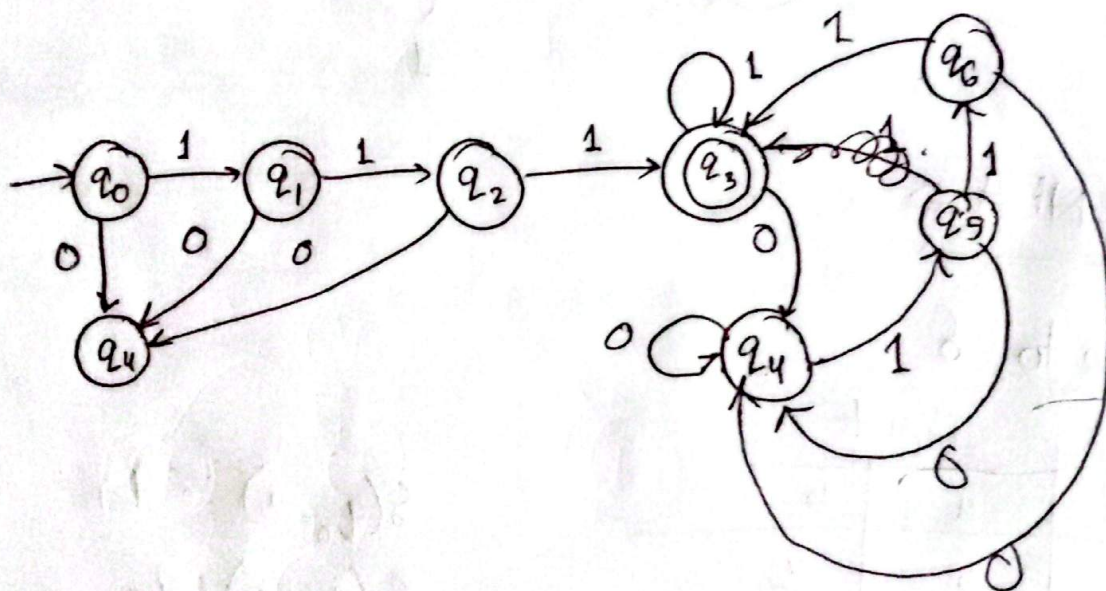
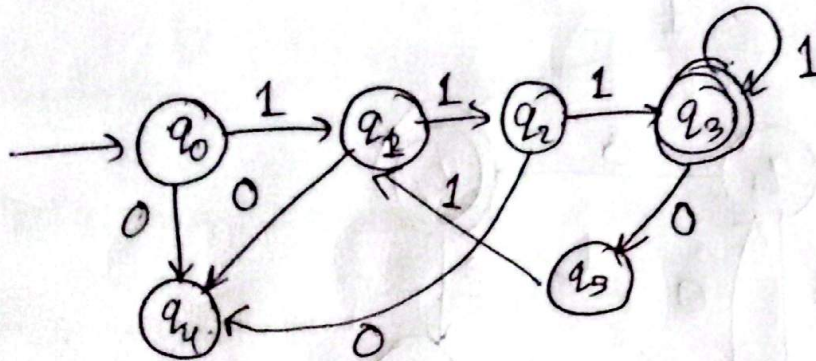
(ii)



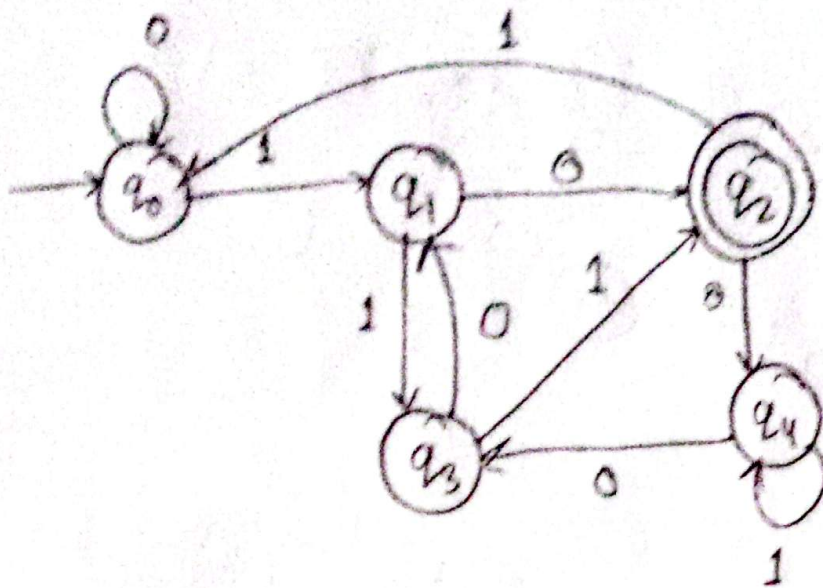
C-7, 10-3 (a-1)

07/06/26

Begin and end with 111



S.Y. 5: 2



~~□ NFA~~

* How to δ

State	Input	
	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$* q_2$	q_2	q_2

Table

$$\begin{aligned} \delta(q_0, a) &= q_1 \\ \delta(q_0, b) &= q_0 \end{aligned}$$

$\hat{\delta} \rightarrow$ Extended transition

$$\hat{\delta}(q_0, \hat{\delta}(q_0, [\text{string upto now}]), [\text{next alphabet}]) = q_i$$

* DFA gives a sequence of traversing states $\rightarrow q_0, q_1, q_2, q_3, \dots$

This traversing is denoted by $\hat{\delta}$

* Using induction,

Basis: $\hat{\delta}(q, \epsilon) = q$

Induction: let, $w = x a$

where a is the last symbol of w
and x is the string consisting of until last symbol

$$\therefore \hat{\delta}(q, w) = \hat{\delta}(\hat{\delta}(q, x), a) \rightarrow \hat{\delta}(q_0, \epsilon) = q_0$$

$$\begin{aligned} \hat{\delta}(q_0, 1) &= \hat{\delta}(\hat{\delta}(q_0, \epsilon), 1) \\ &= \hat{\delta}(q_0, 1) \\ &= q_3 \end{aligned}$$

$$\begin{aligned} \hat{\delta}(q_0, 11) &= \hat{\delta}(\hat{\delta}(q_0, 1), 1) \\ &= \hat{\delta}(q_3, 1) \end{aligned}$$

$$= q_0$$

* The language of a DFA

$A = (Q, \Sigma, \delta, q_0, F)$ denoted by $L(A)$

is defined by

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$

* Quiz on Tuesday
upto DFA

C-8, Q-3 (Q-1.5)

09/07/26

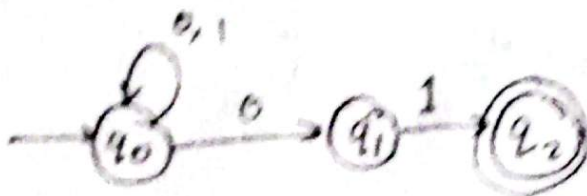
NFA: Non-Deterministic Automata

→ for some state, it can be in several states at once

→ like DFA $\rightarrow A = (Q, \Sigma, \delta, q_0, F)$

if string ends with 01

pseudocode of DFA



	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

* Extended Transition:

Basis: $\delta(q, \varepsilon) = q$

$w = a_1 a_2 \dots a_n$

$$S(q, a) = \{p_1, p_2, \dots, p_r\}$$

$$\therefore \bigcup S(p_i, a) = \{n_1, n_2, n_3, \dots\}$$

$$\hat{\delta}(q_0, \varepsilon) = q_0$$

$$\begin{aligned}\hat{\delta}(q_0, 0) &= \hat{\delta}(\hat{\delta}(q_0, \varepsilon), 0) \\ &= \hat{\delta}(q_0, 0) = \{q_0, q_1\}\end{aligned}$$

$$\begin{aligned}\hat{\delta}(q_0, 00) &= \hat{\delta}(\hat{\delta}(q_0, 0), 0) \\ &= \hat{\delta}(\{q_0, q_1\}, 0) \\ &= S(q_0, 0) \cup S(q_1, 0) \\ &= \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}\end{aligned}$$

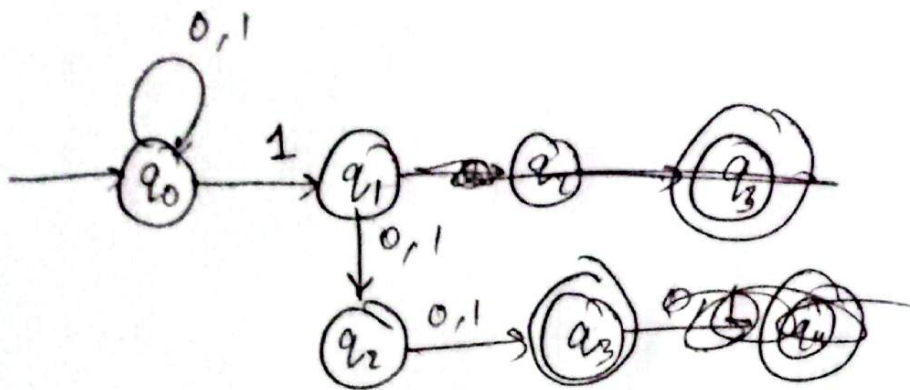
$$\begin{aligned}\hat{\delta}(q_0, 001) &= \hat{\delta}(\hat{\delta}(q_0, 00), 1) \\ &= \hat{\delta}(\{q_0, q_1\}, 1)\end{aligned}$$

$$= \delta(q_0, 1) \cup \delta(q_1, 1)$$

$$= \{q_0\} \cup \{q_2\}$$

$$= \{q_0, q_2\}$$

3rd test 1



Online

Q-9, W-4? Q-1.5

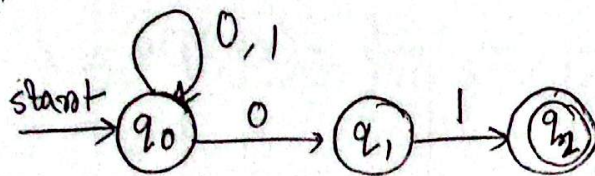
16/07/26

DF NFA to DFA

Let, $N = \{Q_N, \Sigma, \delta_N, q_0, F_N\}$
 $D = \{Q_D, \Sigma, \delta_D, q_0, F_D\}$

where, $Q_D \subseteq Q_N$

* Ends with 1

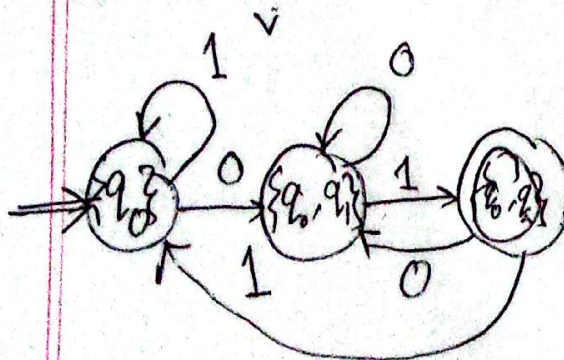


$\therefore Q_N = \{q_0, q_1, q_2\}$

δ table:

state	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_1\}$	$\{q_0, q_1\}$	$\{q_2\}$
$\{q_2\}$	$\{q_0, q_1\}$	$\{q_2\}$

state	0	1
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$



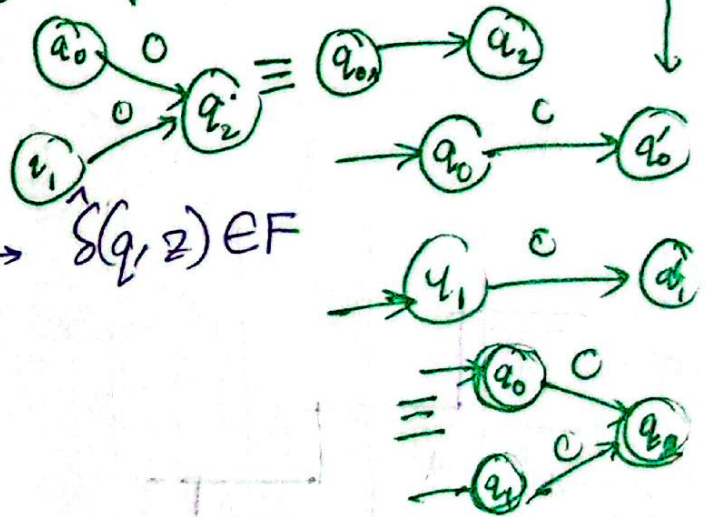
as no new state

DFA minimization: i) if multiple states lead to same

ii) if multiple states have same input to dif similar states

iii) " " " have same output to some state

iii) remove unreachable

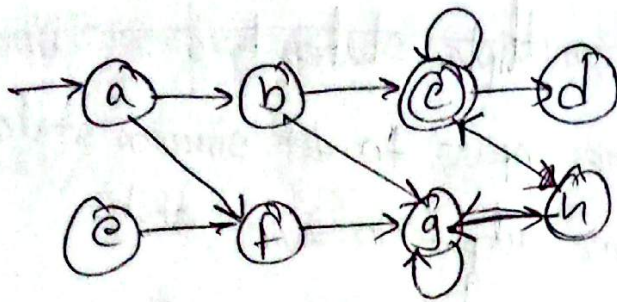


$$\forall z \in \Sigma^*, \delta(p, z) \in F \leftrightarrow \delta(q, z) \in F$$

i.e. iff

Q-10 & 11, W-5, Q-2

21/07/26

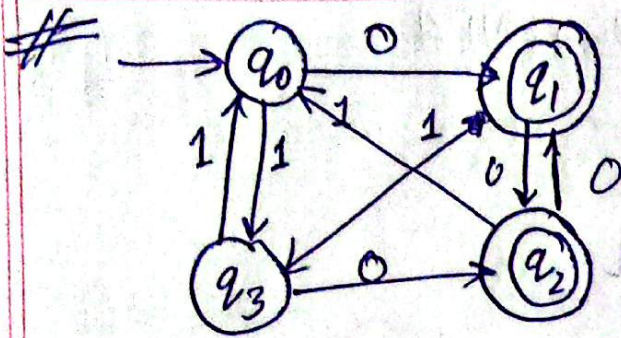


	A	B	C	D	E	F	G	H
A								
B	* ⁽ⁱ⁾							
C	* ⁽ⁱ⁾	* ⁽ⁱ⁾						
D	* ⁽ⁱ⁾		* ⁽ⁱ⁾					
E			* ⁽ⁱ⁾					
F			* ⁽ⁱ⁾					
G			* ⁽ⁱ⁾					
H	(a,e)		* ⁽ⁱ⁾					
	A	B	C	D	E	F	G	H

← This part is unnecessary

* Table Filling Method

- i) Mark final, non-final → no dependency → cannot be merged
- ii) Take each cell and check each transition of pairs
 - ↳ if any pair's transition is marked → mark them
 - else mark the transition with the 00
- iii) At the end → all unmarked are equivalent and can be merged



for future self: don't be careless

non-final non-final are accepted

q_1	*		
q_2	*	✓	
q_3	✓	*	*
	q_0	q_1	q_2

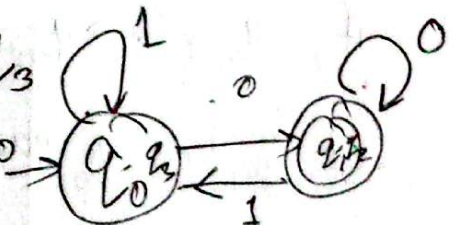
$$\delta(q_3, \epsilon) = q_2$$

$$\delta(q_0, 0) = q_1$$

$$\delta(q_3, 1) = q_3$$

$$\delta(q_0, 1) = q_0$$

∴ unmarked



$$\delta(q_2, 0) = q_1$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_2, 1) = q_0$$

$$\delta(q_1, 1) = q_3$$

∴ unmarked

∴ q_3, q_0 are equivalent

∴ q_2, q_1 " "

ϵ -NFA $\rightarrow$ "easier" than NFA

$$S: Q \times \Sigma = 2^Q$$

and, $Q \times (\Sigma \cup \epsilon) = 2^Q$

$$q_0 \rightarrow q_3$$

if, $q_0 \xrightarrow{\epsilon} q_3$

it implies q_0 can transition to q_3 without any input

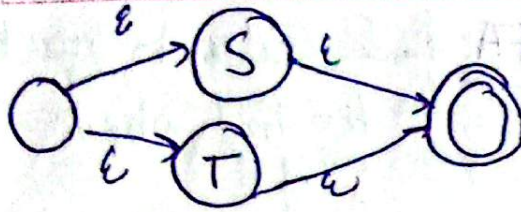
But, q_0 and q_3 may not be equivalent ???

RegEx of variable:

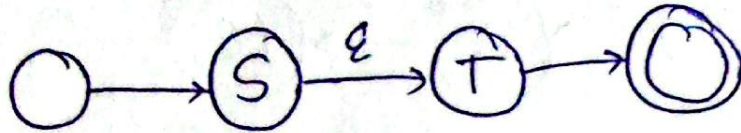
$$R = \left((A-z)^* + (a-z)^* + (-)^* \right) \cdot (0-9)^*$$

$$\left((A-z)^* + (a-z)^* + (-)^* \right) \cdot \left((A-z)^* + (a-z)^* + (-)^* + (0-9)^* \right)$$

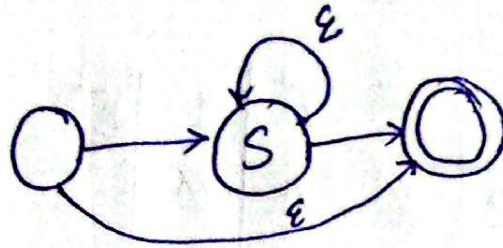
$$* R = S + T$$



$$R = ST$$



$$R = S^*$$

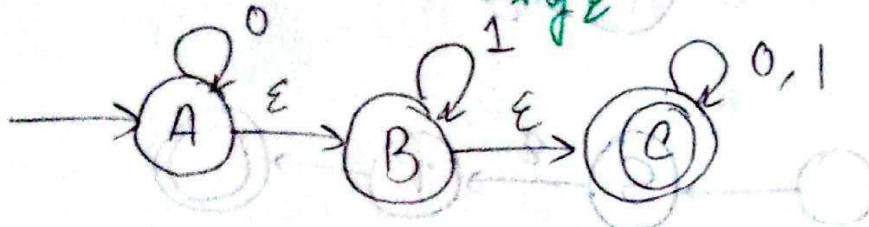


ϵ^* (ϵ closure): Total how many states that can be reached using ϵ from any state

C-12, W-5, (Q-2)

23/07/26

ϵ -NFA to NFA: find states to go to using ϵ
 $\hookrightarrow$ the find where those states to go to using ϵ

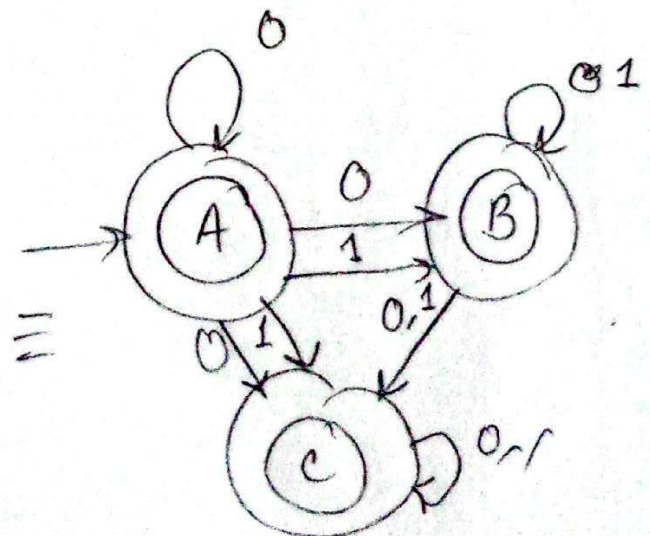


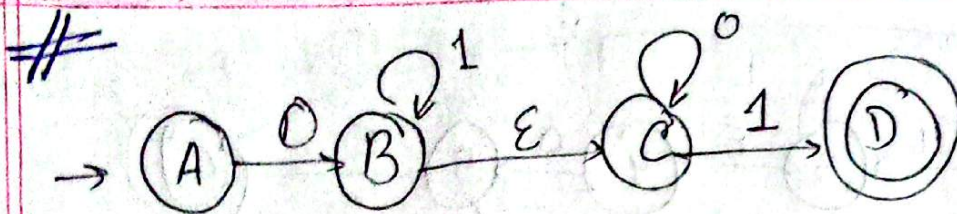
why this?

A	ϵ^*	0	ϵ^*	ϵ^*	1	ϵ^*
	A	A	A, B, C	A	$\emptyset$	
	B	$\emptyset$		B	B	B, C
	C	C	C	C	C	C
B	ϵ^*	0	ϵ^*	ϵ^*	1	ϵ^*
	B	$\emptyset$		B	B	B, C
	C	C	C	C	C	C
C	ϵ^*	0	ϵ^*	ϵ^*	1	ϵ^*
	C	C	C	C	C	C

ϵ^* leads to C
 That is why they are the final state

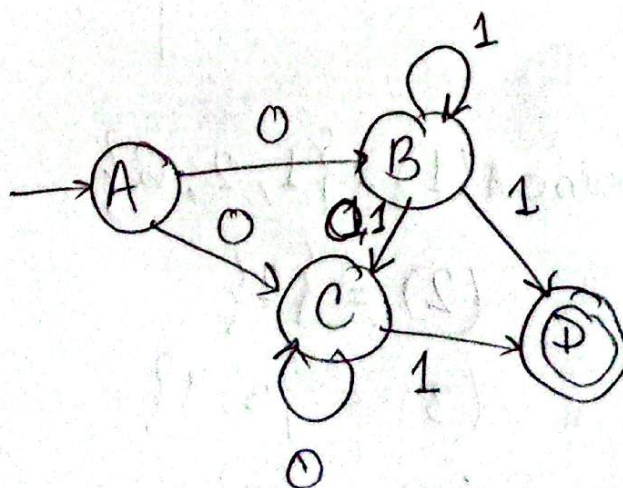
	0	1
$\rightarrow^* A$	$\{A, B, C\}$	$\{B, C\}$
$^* B$	$\{C\}$	$\{B, C\}$
$^* C$	$\{C\}$	$\{C\}$





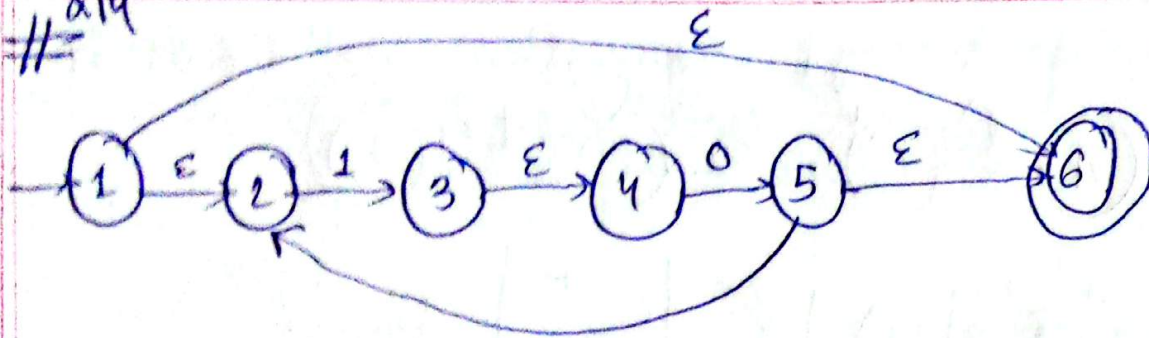
	ϵ^*	0	ϵ^*	ϵ^*	1	ϵ^*
A	A	B	B, C	A	$\emptyset$	
B	B	$\emptyset$		B	B	B, C
	C	C	C	C	D	D
C	C	C	C	C	D	D
D	D	$\emptyset$		D	$\emptyset$	

	0	1
A	{B, C}	$\emptyset$
B	{C}	{B, C, D}
C	{C}	{D}
*D	$\emptyset$	$\emptyset$



C-13 W-6(Q-2)
 #8/14

28/07/26



State	ϵ^*	0	ϵ^*	ϵ^*	1	ϵ^*
1	1 2 6	1 2 $\emptyset$		1 2 6	1 3 $\emptyset$	$\{4\}$
2	1 2	$\emptyset$		2	3	$\{4\}$
3						

ϵ -closure(1) = $\{1, 2, 6\}$

" (2) = $\{2\}$

" (3) = $\{3, 4\}$

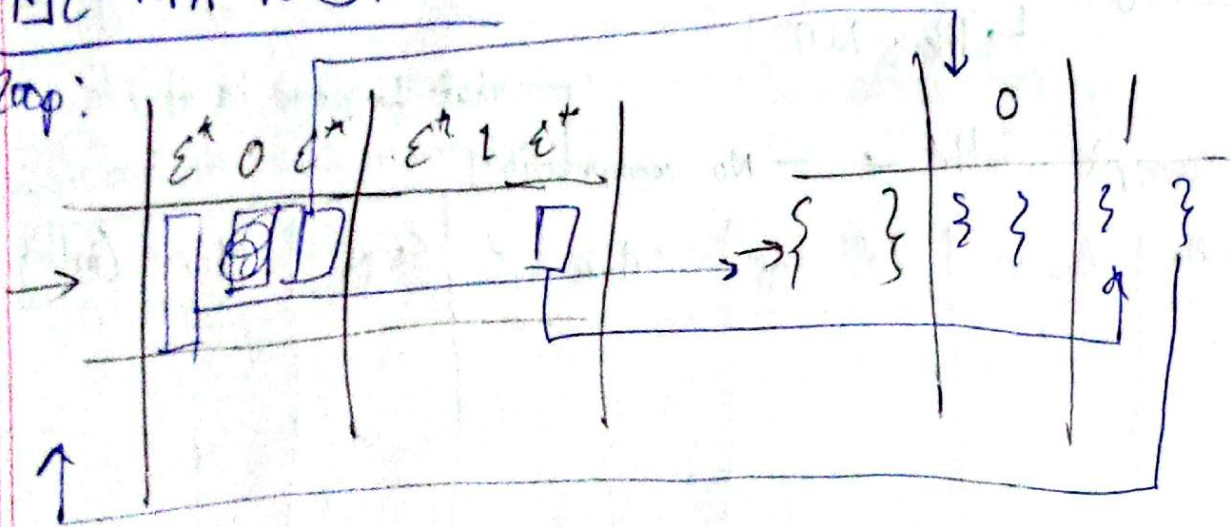
" (4) = $\{4\}$

" (5) = $\{5, 6\}$

" (6) = $\{6\}$

☐ ϵ -NFA to DFA:

loop:



☐ Lexical Analysis:

Scanning $\rightarrow$ remove space
 Lexical Analysis $\rightarrow$ convert stream of chars to stream of token

* delimiters: single $\rightarrow$, , ; , # . . .
 pair $\rightarrow$ " " , { } , () . . .

Symbol Table: Stores imp info — (basically everything)

↳ like LUT

that happens in stack

sample table. ← No computation

Sr. No.	Name	Id type	Value type	Scope	More (info)

* Kept usually as an unordered list

usually Linear

BST

Hash Table

* Times → i) insert()
ii) lookup()

Syntax Analysis → Hey that's us ☹️

↳ Generates Tree

Context Free Grammar

* To reach leaf node, we need grammar

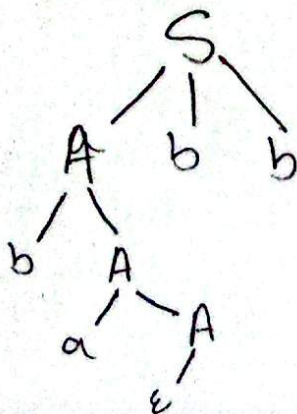
$G = (V, T, P, S)$; $V =$ finite set of variables or non-terminal symbols (we use capital letters) $\left. \begin{array}{l} T = \text{terminal symbols} \\ \text{(we use lowercase letters)} \end{array} \right\} V \cap T = \emptyset$
 $P =$ Production → rules
 $S =$ start variable → $S \in V$

$A \rightarrow \alpha$ } A produces α
↑ ↑ ↑
what produces what

$S \rightarrow Abb$
 $A \rightarrow Aa$
 $A \rightarrow \epsilon$
 $A \rightarrow abA$ } $A \rightarrow Aa | \epsilon | abA$

* * We use Grammar Tree to verify RegEx.

↳ $ba b b$
 $s \rightarrow \overset{A}{b} a b b$
→ $b A b b$
→ $b a A b b$
→ $b a \epsilon b b$
→ $b a b b$



∴ verified

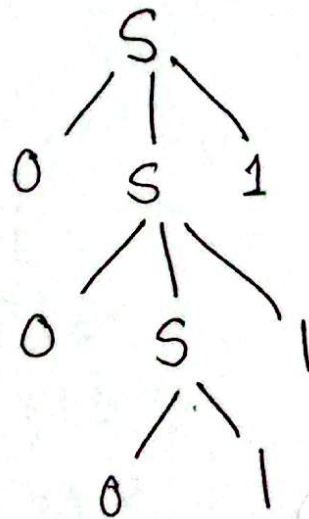
C-15, W-6 (Q-2.5)

30/07/26

$L_1 = \{0^n 1^n \mid n > 0\} \rightarrow G = (\{S\}, \{0, 1\}, \{S \rightarrow 01, S \rightarrow 0S1\}, \{S\})$

$S \rightarrow 01 \mid 0S1$

for 000111



alt meth: $S \xrightarrow{2m} 0 S 1$
 $\rightarrow 0 0 S 1 1$
 $\rightarrow 0 0 0 1 1 1$

* mention if 2m or nm

$$\# E \rightarrow I \mid E + E \mid E * E \mid (E)$$

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I$$

$$\# \text{String} = a + (a * b00)$$

$$E \xrightarrow{?m} E + E$$

$$\rightarrow I + E$$

$$\rightarrow a + E$$

$$\rightarrow a + (E)$$

$$\Rightarrow a + (E * E)$$

$$\rightarrow a + (OI * E)$$

$$\rightarrow a + (a * E)$$

$$\rightarrow a + (a * I)$$

$$\rightarrow a + (a * IO)$$

$$\rightarrow a + (a * IOO)$$

$$\rightarrow a + (a * b00)$$

* Unique tree

↳ but if not → ambiguous grammar

$E \rightarrow E + E \mid E * E \mid (E) \mid x \mid y \mid z$

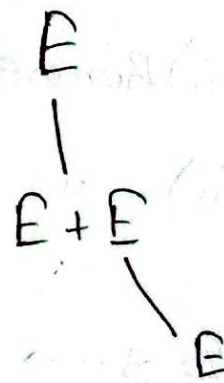
1. $E \xrightarrow{2m} E + E$

→ $E + E + E$

→ $x + E + E$

→ $x + y + E$

→ $x + y + z$



2. $E \rightarrow x + E$

→ $x + E + E$

→ $x + y + E$

→ $x + y + z$

∴ Ambiguous

C-16, W-7, Q-2.5

04/08/26

Simplification of CFG:

i) Remove Useless Variables

- Non Generating variable
- Non Reachable variables

ii) Remove ϵ -productions $\rightarrow$ ones except starting node

iii) " Unit "

$S \rightarrow AB|CA$

$A \rightarrow a$

$B \rightarrow BC|AB \rightarrow$ Non generating $\rightarrow$ Recursive

$C \rightarrow aB|b$

$\therefore S \rightarrow CA$

$A \rightarrow a$

$C \rightarrow b$

$S \rightarrow aAa|aB$

$A \rightarrow aS$

$B \rightarrow aBa|b$

$C \rightarrow aab \rightarrow$ unreachable

$\left\{ \begin{array}{l} S \rightarrow aAa|aB \\ A \rightarrow aS \\ B \rightarrow aBa|b \end{array} \right.$

$$S \rightarrow Bac | bTC | ba$$

$$T \rightarrow aB | TB$$

$$B \rightarrow aTB | bBC$$

$$C \rightarrow TBc | aBc | ac$$

1. Non generative

2. Non reachable

1

$$\therefore S \rightarrow \cancel{Bac} ba$$

$$C \rightarrow aC$$

2

$$\therefore S \rightarrow ba$$

Remove ϵ -production: $\rightarrow$ Find nullable variables
 $\rightarrow$ Remove all ϵ -productions by replacing some production

$$\left. \begin{array}{l} S \rightarrow aA | ab \\ A \rightarrow bA | \epsilon \end{array} \right\} \begin{array}{l} S \rightarrow aA | ab | a \\ S \rightarrow aA | ab | a \\ A \rightarrow \cancel{bA} | b \end{array}$$

$\rightarrow$ replace ϵ with variations of ϵ

$\downarrow$
 we replace all productions with ϵ -based

$$\begin{array}{l} S \rightarrow aA \rightarrow ab \\ S \rightarrow aA \rightarrow abA \\ S \rightarrow aA \rightarrow ab\epsilon \\ S \rightarrow aA \rightarrow ab \end{array} \quad \rightarrow ab$$

$S \rightarrow A|B|C$ $\rightarrow$
 $A \rightarrow aAa|B$ $\rightarrow \epsilon$
 $B \rightarrow bB|bb$
 $C \rightarrow aCa|D$ $\rightarrow$
 $D \rightarrow baD|abD|\epsilon$ $\rightarrow$
 $\downarrow$ $\rightarrow$ $ba|ab$

Nullable

$\therefore S \rightarrow A|B|C|\epsilon$
 $A \rightarrow aAa|B$
 $B \rightarrow bB|bb$
 $C \rightarrow aCa|D|aaa$
 $D \rightarrow baD|abD|ba|ab$

$S \rightarrow ABCD$
 $A \rightarrow a$
 $B \rightarrow d|\epsilon$
 $C \rightarrow ED|\epsilon$
 $D \rightarrow BC$
 $E \rightarrow b$

$S \rightarrow ABCD|ABCD|ABCD|ABCD|$
 $A \rightarrow a$
 $B \rightarrow d$
 $C \rightarrow ED|BC|BC|BC|BC|$
 $D \rightarrow BC|B|C$
 $E \rightarrow b$

* Removal of Unit Production:

$S \rightarrow A | B | C$

$A \rightarrow aa | B$

$B \rightarrow bb | C$

$C \rightarrow cc | A$

$S \rightarrow A \rightarrow B \rightarrow C$

$\therefore B$ and C can be replaced with A

$\therefore S \rightarrow A | A | A$

$A \rightarrow aa | A$

$A \rightarrow bb | A$

$A \rightarrow cc | A$

$S \rightarrow A$

$A \rightarrow aa | bb | cc$

$S \rightarrow aa | bb | cc$

$S \rightarrow A | Bb$

$A \rightarrow c | a$

$B \rightarrow aBa | b$

$C \rightarrow aSa$

show steps

~~$S \rightarrow A \rightarrow C \rightarrow$~~

$S \rightarrow c | a | Bb$

$\rightarrow aSa | a | Bb$

$A \rightarrow c \rightarrow aSa$

$B \rightarrow aBa | b$

$\therefore S \rightarrow aSa | a | Bb$

$A \rightarrow aSa | a$

$B \rightarrow aBa | b$

$C \rightarrow aSa$

$S \rightarrow aSa | a | Bb$

$B \rightarrow aBa | b$

CFL from CFG

CNF (Chomsky Normal Form),

We apply this after CFG

1. Arrange all vars to be of 2 or more vars
2. Expresses terminal with just the other vars

$$\# S \rightarrow ASB \mid AB$$

$$A \rightarrow aAS \mid aA \mid a$$

$$B \rightarrow sbs \mid bs \mid os \mid b \mid aAS \mid aA \mid a \mid bb$$

$$S \rightarrow ASB \mid AB$$

$$A \rightarrow CAS \mid BA \mid a$$

$$B \rightarrow SDS \mid DS \mid SD \dots$$

$$C \rightarrow a$$

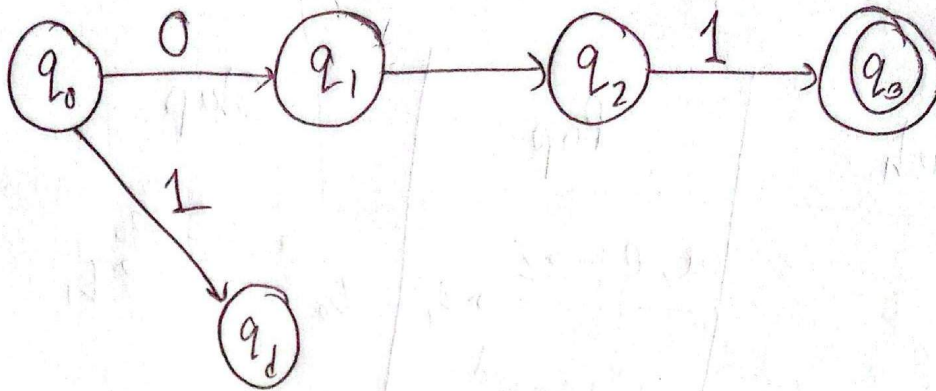
$$D \rightarrow b$$

[Just check the slide
man,
TS is too complicated
to write]

00-18, W-8, Q-3

18/08/26

$\#0^n 1^n; n > 0$



This cannot be done normally

This is why its called Finite Automata

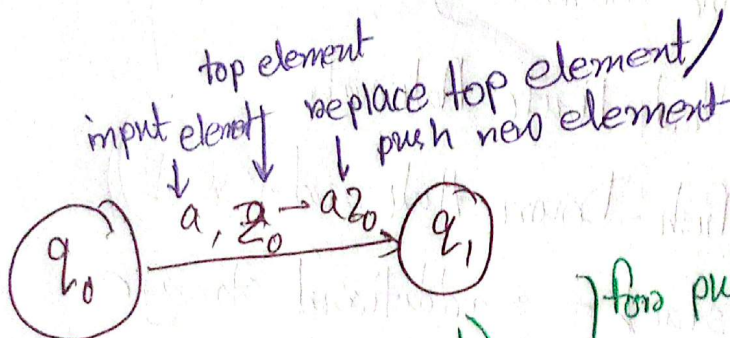
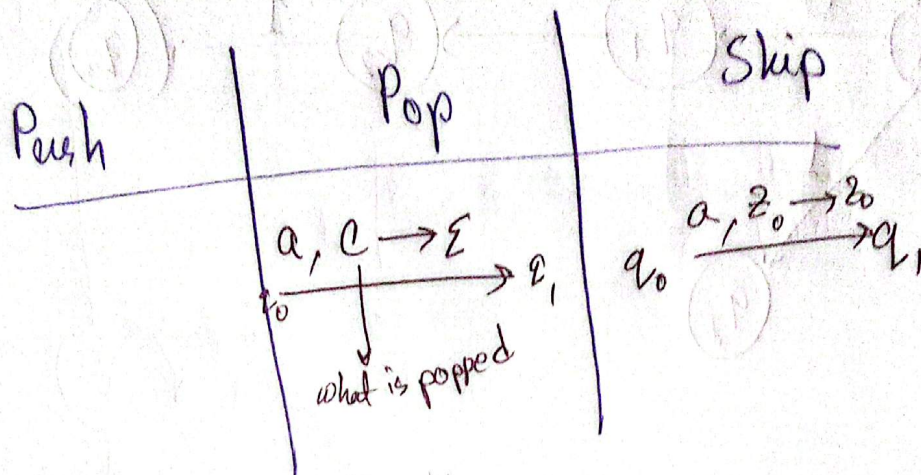
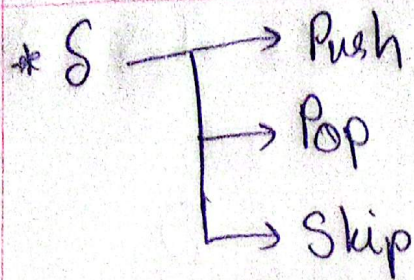
we switch to Push-Down Automata

↳ we add a stack → additional storage

$\{Q, \Sigma, \delta, q_0, F, \Gamma, z_0\}$

Set of Stack
symbol

Bottom/Initial
state



if (input element)
replace stack.top()
with new push

for push, replace top with sth new & prev_{stack}
pop, replace top with ϵ
skip, replace top with top

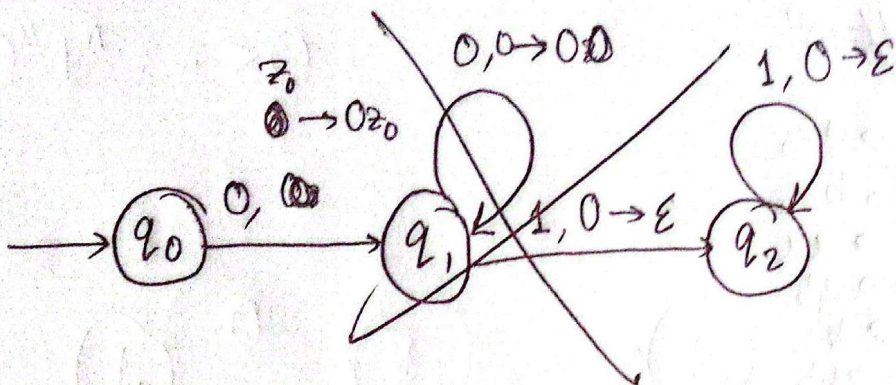
$\neq a z_0 \neq$

z_0
a

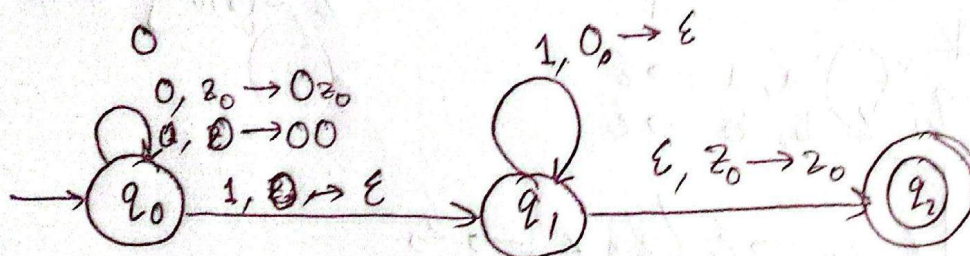
$a z_0 =$

a
z_0

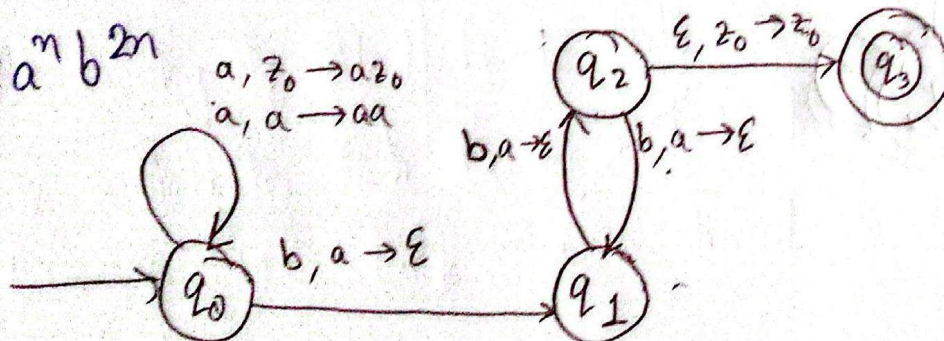
why? I have no idea



$\therefore 0^n 1^n, n > 0$



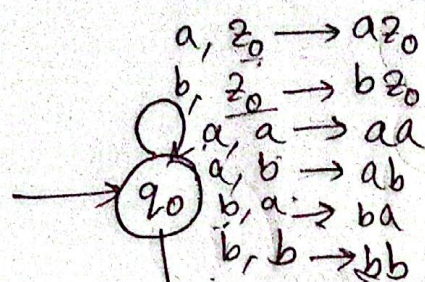
$a^n b^{2n}$



99992

PDA cannot solve multivariable (≥ 2) problems
We need Turing test for it

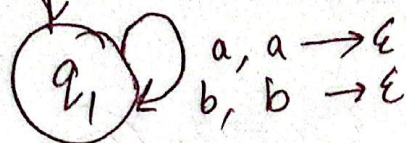
~~# ab# ba~~ with w^n



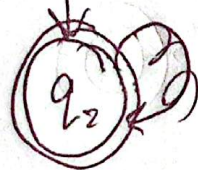
$\#, a \rightarrow a$
 $\#, b \rightarrow b$

$\#, z_0 \rightarrow z_0$

iff the question states ϵ string is accepted



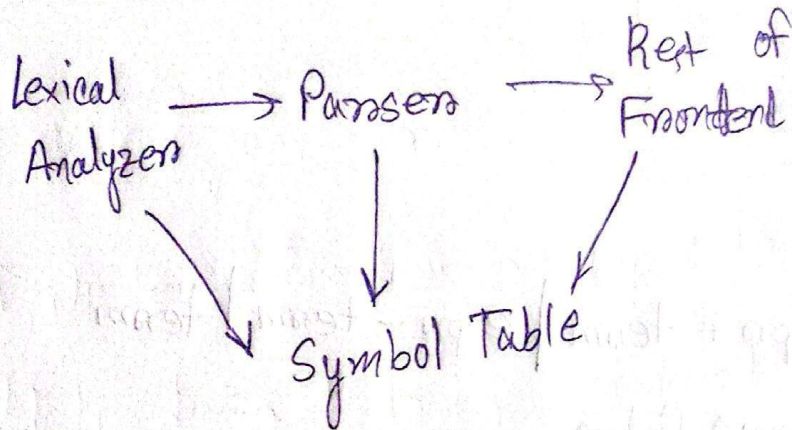
$\epsilon, z_0 \rightarrow \epsilon / \epsilon, z_0 \rightarrow z_0$



Q-19, 20, Q-W-9, Q-2.5

25/08/26

Syntax Analysis → to generate parse tree
from token
(from lexical analyzer)



* RE can be expressed by Grammars
but not vice-versa

↓ Post-fix but prior when left-recursive
☐ Parse Tree: [I am having an existential crisis
please check slides.

It's pretty straightforward anyways]

☐ Token vs Terminals: Token → < token name, attr >

Precedence of Op:

()

* / %

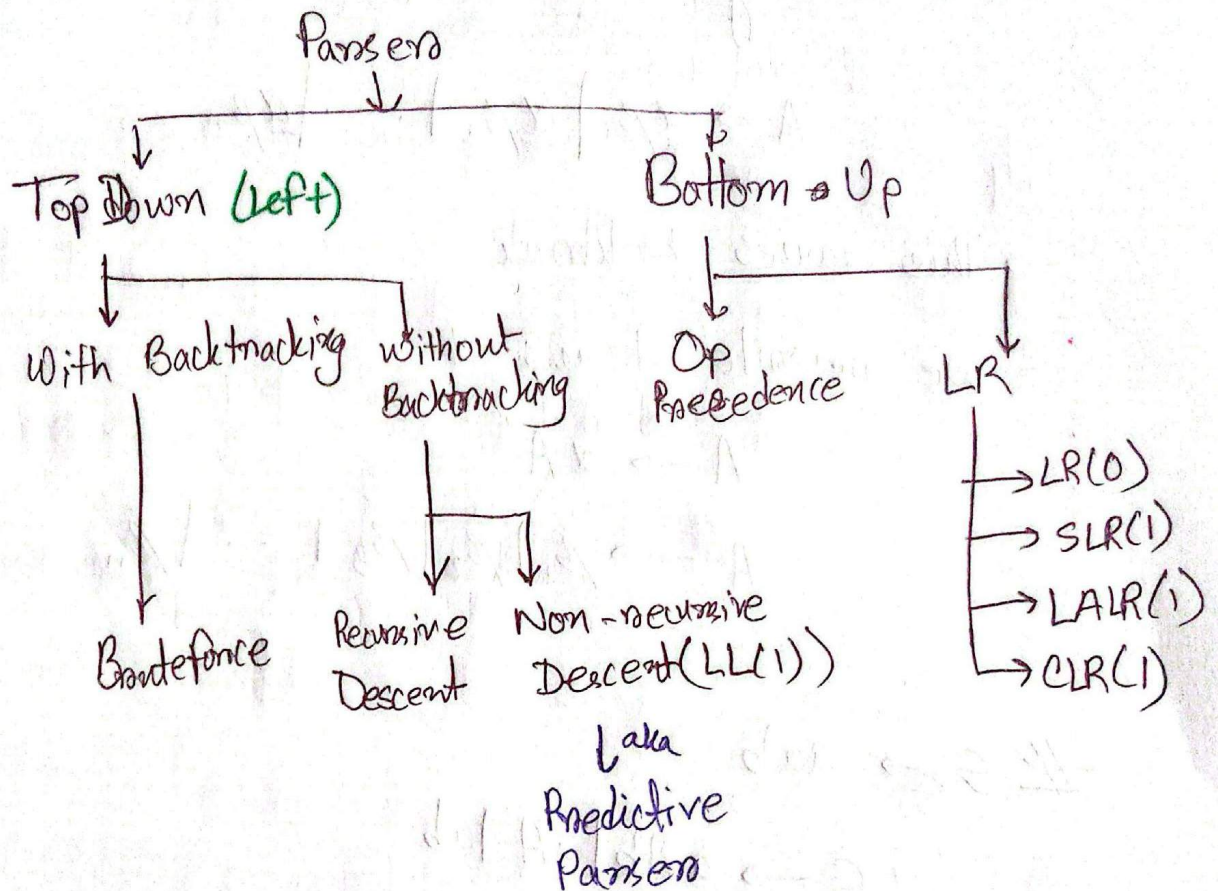
+ -

=

* $\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$

$\text{term} \rightarrow \text{term} * \text{factor}$

Type of Parsers:



LL(1) : ~~input is parsed~~

- i) scanned from left to right
- ii) Performs left most
- iii) Elimination of left recursion
- iv) " " left factoring
- v) " " Ambiguous grammar

* Left Factoring: if all left have same terminal

$$A \rightarrow \underline{\alpha}\beta_1 \mid \underline{\alpha}\beta_2 \mid \dots \mid \underline{\alpha}\beta_n$$

→ this causes backtrack

∴ we rewrite it as:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$$

~~S~~ aab

$$S \rightarrow aAb \mid aA \mid ab$$

$$A \rightarrow bAc \mid ab$$

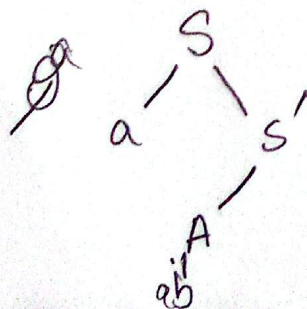
$$\therefore S \rightarrow aS'$$

$$S' \rightarrow Ab \mid A \mid b \rightarrow \cancel{S''b} \mid \cancel{AS''} \mid \cancel{Ab}$$

$$A \rightarrow bAc \mid ab$$

For

$$\therefore aab, \rightarrow S \rightarrow aS' \rightarrow \cancel{aAb} \rightarrow \cancel{aAS''} \rightarrow aab$$



* Recursion: calls itself

↳ in this case a var that calls itself

→ Left Recursion

$$A \rightarrow A\alpha \mid \beta$$

↓ Direct

Replace with

$$A \rightarrow A\alpha \mid \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

→ Right Recursion

$$A \rightarrow \alpha A \mid \beta$$

$$\text{Inlined} \leftarrow \begin{aligned} S &\rightarrow A A \mid 0 \\ A &\rightarrow S S \mid 1 \end{aligned}$$

$$\# T \rightarrow T * E \mid F$$

↓

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$\# \text{expr} \rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$$

$$\text{expr} \rightarrow \text{term expr}' \mid \text{term expr}''$$

$$\text{expr}' \rightarrow + \text{term expr}' \mid - \text{term expr}' \mid \epsilon$$

$$\text{expr}'' \rightarrow - \text{term expr}'' \mid \epsilon$$

Compiler Design

First & Follow : Pre-phase of parser

First: Takes First Terminal of productions (of all variables for " ")

$$A \rightarrow abc \mid bB \quad \left| \quad \begin{array}{l} A \rightarrow BcD \mid \epsilon \\ B \rightarrow cD \mid (A) \end{array} \right.$$

$$\therefore \text{First}(A) = \{a, b\} \quad \left| \quad \begin{array}{l} A \rightarrow \\ \therefore \text{First}(A) = \{c, (, \epsilon\} \end{array} \right.$$

→ First of S will be $\{\$ \}$

$$\# E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

First	First
E	{(, id}
E'	{+, ε}
T	{(, id}
T'	{*, ε}
F	{(, id}

Q-22, W-9, Q-3.5
set of

Follow: ~~set~~ terminal after the variable

→ if non-terminal → take FIRST(non-terminal)

→ if finds itself, take FOLLOW(non-terminal)

→ be careful of ϵ

$$\therefore A \rightarrow BA \epsilon \mid aAB$$

$$B \rightarrow cA \mid d$$

$$\therefore \text{FOLLOW}(A) = \{c, \text{FIRST}(B), \text{FOLLOW}(B), \$\}$$

$$\# A \rightarrow aBC \mid Ab \mid d$$

$$B \rightarrow (c) \mid Aa$$

$$C \rightarrow + \mid \epsilon$$

$$\text{First}(A) = \{a, d\}$$

$$\text{First}(B) = \{(, \text{First}(A)\}$$

$$= \{(, a, d\}$$

$$\text{First}(C) = \{+, \epsilon\}$$

$$\therefore \text{Follow}(A) = \{b, a, \epsilon\}$$

→ start symbol

$$\begin{aligned} \therefore \text{Follow}(B) &= \{\text{First}(C)\} = \{+, \epsilon\} \\ &= \{+, \text{Follow}(A)\} = \{+, b, a, \epsilon\} \end{aligned}$$

only for ϵ

$$\therefore \text{Follow}(C) = \{\epsilon,)\} = \{\text{Follow}(A),)\} = \{b, a, b,)\}$$

C-22, W-9, Q-3.5

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

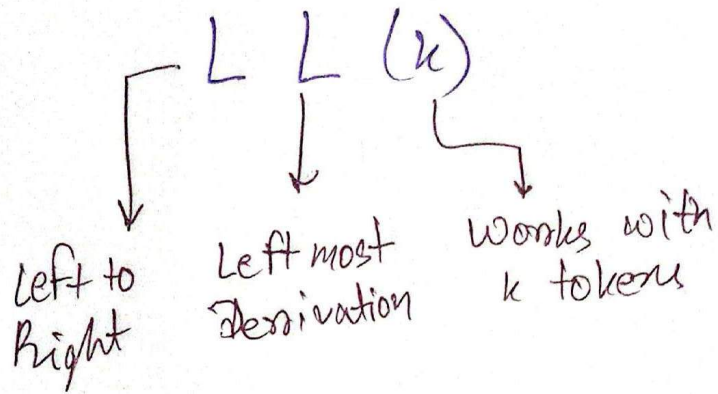
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

	First	Follow
E	{(, id}	{), \$}
E'	{+, ε}	{ε} = {Follow(E'), Follow(E)} = {), \$}
T	{(, id}	{First(E')} = {+, ε} = {+, Follow(E')}
T'	{*, ε}	= {+, (, id} = {+, \$, (}
F	{(, id}	{ε} = {Follow(T)} = {+, (, \$}
		{First(T')} = {*, ε} = {*, Follow(T), Follow(T')}
		= {*, +, \$, (}

LL(1) Parser



	Input Symbol			
Non-Terminal				

Fill Parsing Table
Take Stack
Make Tree